임베딩: 벡터와 유사도

왜 텍스트를 숫자로 바꿔야 하는가?

컴퓨터가 다룰 수 있는 것은 오직 숫자뿐!

사람: "경제 성장 정책 발표" → 곧바로 이해
컴퓨터: "경제 성장 정책 발표" → ???

해결: 텍스트 → 숫자 리스트(벡터)로 변환!

핵심 질문:
같은 문장 3개를 세 가지 방법으로 벡터화하면 어떤 차이가 생길까?

→ **One-hot** → **TF-IDF** → **임베딩(밀집 벡터)** **순서로 비교!**

예시 문장 3개

문장 A
"경제 성장 정책 발표"
경제 관련 뉴스

문장 B
"경제 침체 대책 논의"
경제 관련 뉴스

문장 C
"야구 선수 홈런 기록"
스포츠 뉴스

A ≈ B (비슷한 주제)

A ≠ C (다른 주제)

이 3개 문장을 One-hot → TF-IDF → 임베딩 세 가지 방법으로 벡터화
A와 B는 주제가 비슷하고, C는 완전히 다른 주제.

각 방법이 이 차이를 얼마나 잘 포착할까?

전체 어휘 (11개): 경제 성장 정책 발표 침체 대책 논의 야구 선수 홈런 기록 → Vocabulary

방법 ① One-hot 인코딩

One-hot: 같은 문장을 벡터로

어휘:	경제	성장	정책	발표	침체	대책	논의	야구	선수	홈런	기록
문장A	1	1	1	1	0	0	0	0	0	0	0
문장B	1	0	0	0	1	1	1	0	0	0	0
문장C	0	0	0	0	0	0	0	1	1	1	1

문제: A와 B는 겹치는 단어가 "경제" 1개뿐 → 유사도가 낮게 나옴
"성장≈침체", "정책≈대책" 같은 의미적 유사성을 전혀 포착 못함!

A·B 코사인 유사도 = 0.25 | A·C 코사인 유사도 = 0.00 → A와 B도 별로 안 비슷해 보임

원리: 해당 단어가 있으면 1, 없으면 0 (벡터 차원 = 어휘 수, 11)

https://wikidocs.net/192729

방법 ② TF-IDF

TF-IDF: 같은 문장에 가중치를 부여

어휘:	경제	성장	정책	발표	침체	대책	논의	야구	선수	홈런	기록
문장A	0.10	0.40	0.40	0.40	0	0	0	0	0	0	0
문장B	0.10	0	0	0	0.40	0.40	0.40	0	0	0	0
문장C	0	0	0	0	0	0	0	0.40	0.40	0.40	0.40

개선: "경제"는 A,B 모두 등장 → IDF 낮음
"성장","정책" 등은 A에만 → IDF 높음

여전한 한계:
"성장≈침체", "정책≈대책" 의미 유사성 모름

A·B 코사인 유사도 = 0.02 | A·C = 0.00 → One-hot보다 약간 나아졌지만 여전히 낮음

원리: TF(빈도) × IDF(희소성) = 중요도 가중치. 그러나 여전히 희소 벡터 (11차원)

https://wikidocs.net/31698

방법 ③ 임베딩 (밀집 벡터)

임베딩(밀집 벡터): 같은 문장, 의미까지 포착!

문장A	0.82	-0.31	0.55	0.12	-0.67	... (768차원)
문장B	0.79	-0.28	0.51	0.09	-0.63	... (768차원)
문장C	-0.15	0.72	-0.44	0.88	0.33	... (768차원)

A와 B의 벡터 패턴이 비슷!

"성장"과 "침체"는 다른 단어이지만
"경제 맥락" 이라는 의미가
벡터에 이미 반영되어 있음

C는 완전히 다른 패턴!

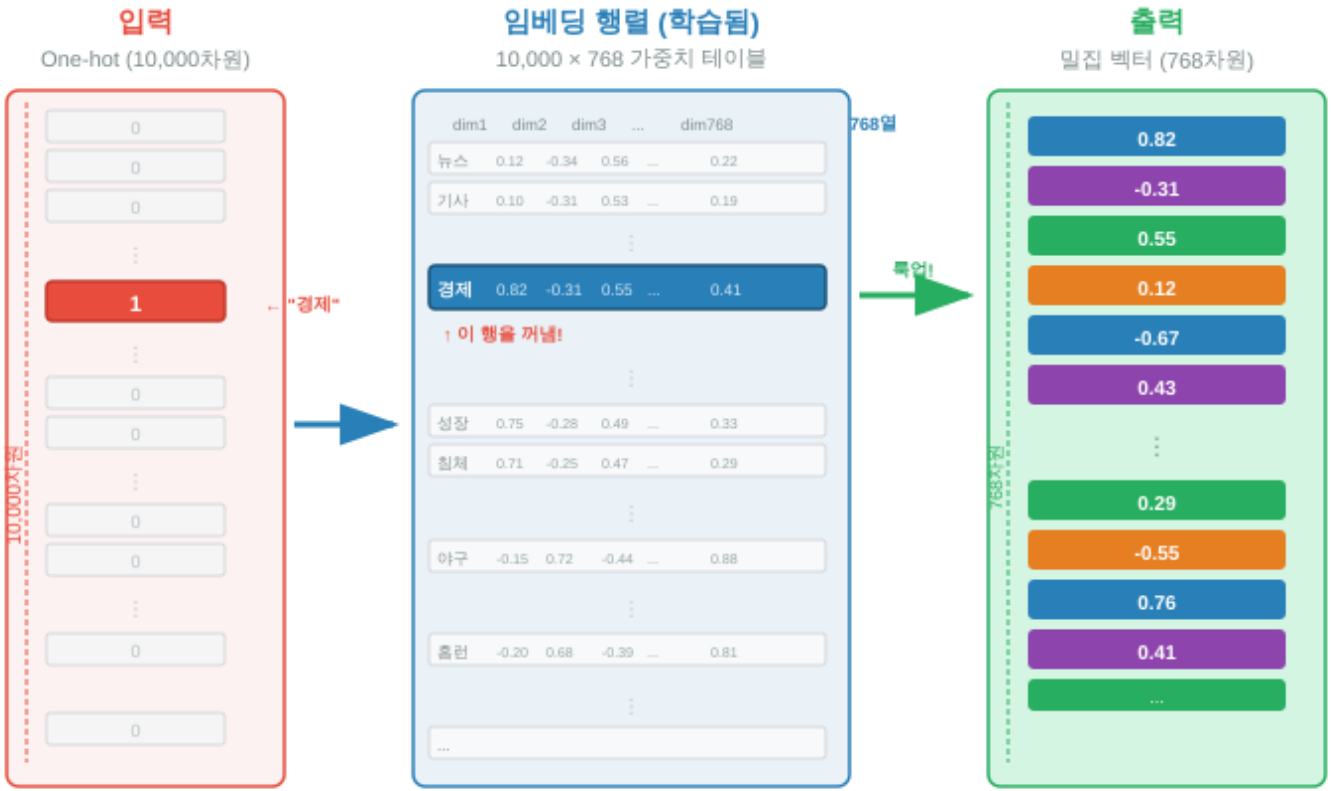
"야구"는 "경제"와
전혀 다른 벡터 방향

A·B 코사인 유사도 ≈ 0.85 (높음!) | A·C ≈ 0.10 (낮음)
→ 단어가 달라도 의미가 비슷하면 유사도가 높다! 🎯

Sentence-BERT: 문장 → 768개 숫자 리스트 (9주차에 내부 구조 학습)

분포 가설: "비슷한 맥락에서 쓰이는 단어는 비슷한 의미를 가진다"
→ 대량 텍스트에서 패턴 학습 → 768차원 밀집 벡터 → 단어가 달라도 의미가 비슷하면 가까운 벡터!

10,000차원 → 768차원: 임베딩 행렬이란?



핵심 원리

One-hot에서 1인 위치 =
행렬에서 해당 행을 꺼냄 (룩업)

차원 축소 효과

10,000차원 → 768차원
(93% 축소, 의미는 보존!)

그런데...

이 768개 숫자는
어떻게 정해지는 걸까?
→ 다음 장에서!

"경제" → One-hot [0,0,...,1,...,0] (10,000칸) → 임베딩 행렬에서 해당 행 꺼냄 → [0.82, -0.31, 0.55, ...] (768칸)

768개 숫자는 어떻게 정해지는가? — 임베딩 학습 과정

STEP 1
대량 텍스트 수집

정부가 경제 성장 정책을 발표...

경제 침체 우려 속 대책 논의...

야구 시즌 개막, 선수 활약...

수억 개의 문장
(위키, 뉴스, 책 등)

STEP 2
빈칸 맞추기 과제 생성

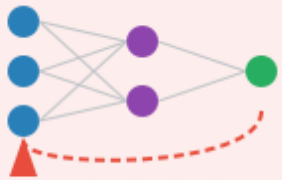
"정부가 ??? 성장 정책을 발표"

정답: "경제"

→ "경제"는 "성장", "정책", "정부" 주변에 자주 등장

→ 주변 단어 패턴으로 의미 학습!

STEP 3
틀리면 가중치 수정 (반복!)



예측:
"정치" ✗

오답 → 가중치 수정!

수억 번 반복하면
"경제" 주변에
"성장","정책","침체"
등이 나오는 패턴을
가중치에 기록!

STEP 4 학습 완료 후 — 임베딩 행렬에 의미가 담긴다!

학습 전: 랜덤 숫자

경제	0.51	-0.87	0.23	...	-0.14
성장	-0.33	0.72	-0.61	...	0.48
야구	0.92	0.04	-0.77	...	0.35

→ 경제≠성장≠야구 (제각각!)

의미 관계 없음 ✗

수억 번 학습!

학습 후: 의미가 담긴 숫자

경제	0.82	-0.31	0.55	...	0.41
성장	0.75	-0.28	0.49	...	0.33
야구	-0.15	0.72	-0.44	...	0.88

비슷한 패턴! 완전 다른 패턴

→ 경제 ≈ 성장 (비슷한 행)

의미 관계 담김 ✓✓

요약: 학습이란?

① 처음에는 랜덤 숫자

② "빈칸 맞추기"를 수억 번

③ 틀릴 때마다 숫자 조정

④ 같은 맥락의 단어는 점점 비슷한 벡터로!

→ 의미 = 사용 패턴!

<부록> 워드투벡터(Word2Vec)

중심 단어 주변 단어

	중심 단어	주변 단어
The fat cat sat on the mat	[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
The fat cat sat on the mat	[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
The fat cat sat on the mat	[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
The fat cat sat on the mat	[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
The fat cat sat on the mat	[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1]
The fat cat sat on the mat	[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

x_{cat} × $W_{V\times M}$ = V_{cat}

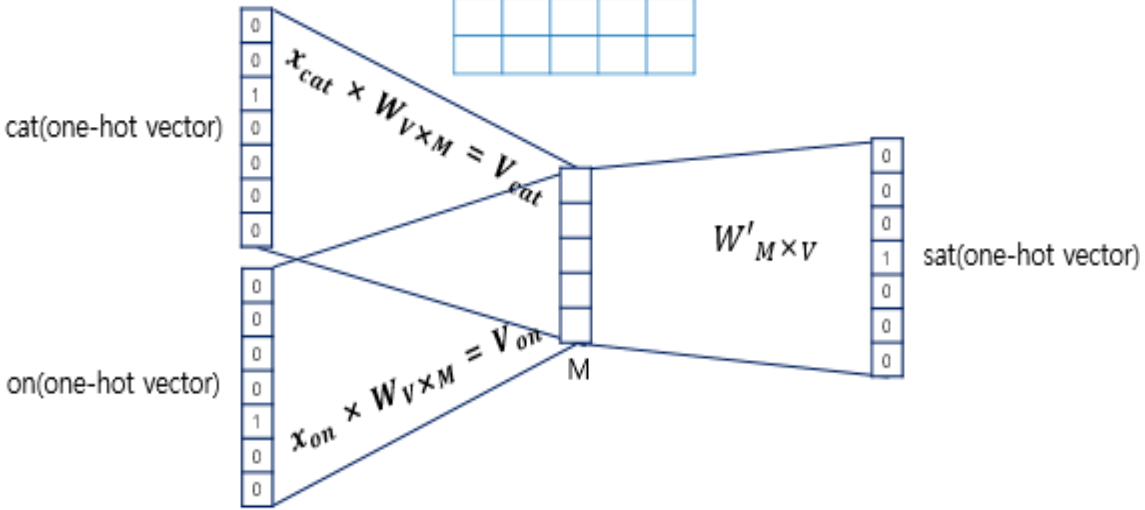
0.5	2.1	1.9	1.5	0.8
0.8	1.2	2.8	1.8	2.1
2.1	1.8	1.5	1.7	2.7

0	0	1	0	0	0	0

× =

2.1	1.8	1.5	1.7	2.7

lookup table



https://wikidocs.net/22660

비교: 같은 문장인데 유사도가 이렇게 달라진다!

같은 문장 A·B 유사도 비교

One-hot
0.25
"경제"만 겹침 → 낮음
의미 관계 ✗

TF-IDF
0.02
"경제" IDF 낮아 기여 미미
의미 관계 △

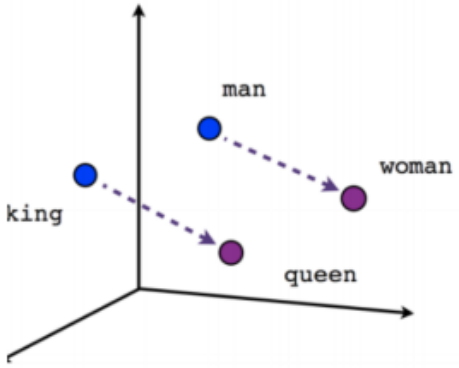
임베딩
0.85
의미적 패턴 일치!
의미 관계 ✓✓

같은 문장인데 방법에 따라 유사도가 이렇게 달라진다!

Word2Vec의 유명한 예시

king − man + woman = queen

→ 벡터 공간에서 '성별'이라는 의미적 관계가 방향으로 인코딩됨
→ "성장"과 "침체"는 다른 단어지만, "경제 맥락"이라는 의미가 벡터에 이미 반영!



이미지도 벡터로: 멀티모달 임베딩

텍스트뿐 아니라 이미지, 음성, 영상도 모두 벡터로 변환 가능!

이미지 임베딩

이미지 → 딥러닝 모델 → 벡터

비슷한 이미지 → 비슷한 벡터
다른 이미지 → 먼 벡터

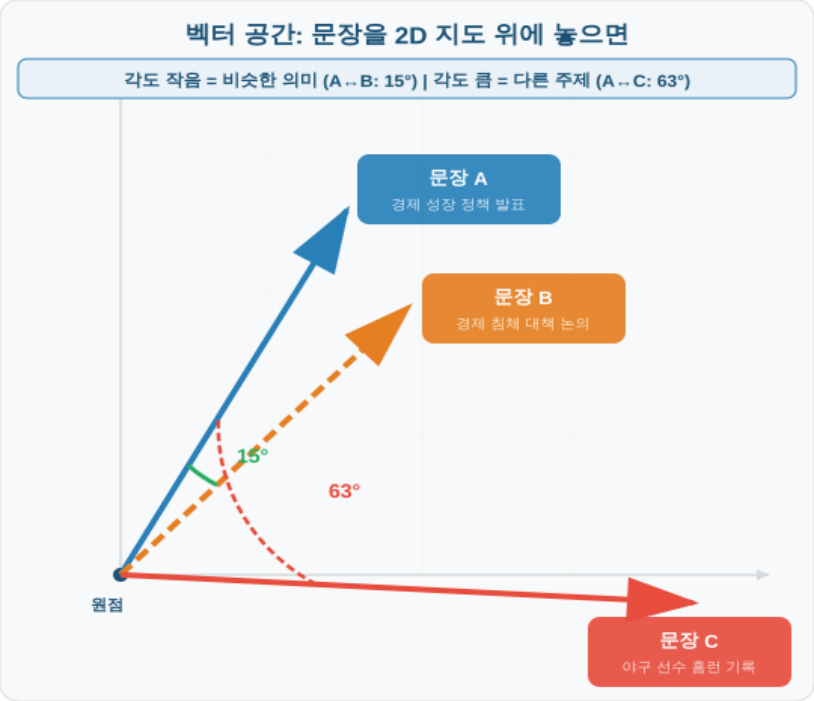
CLIP (OpenAI, 2021)

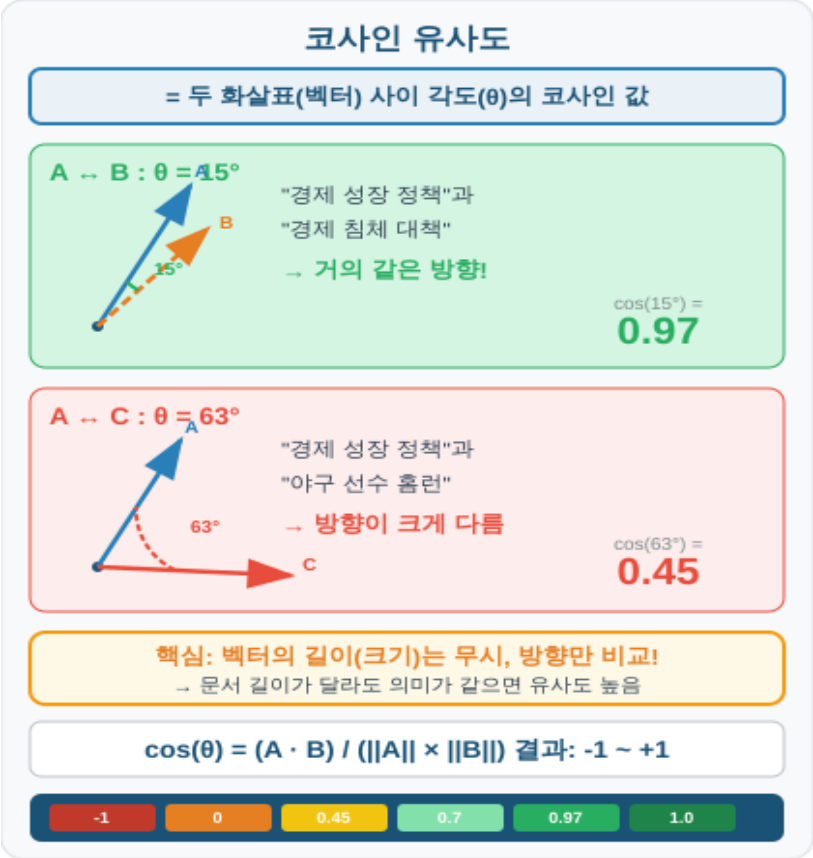
텍스트와 이미지를
같은 벡터 공간에 배치!

→ 텍스트로 이미지 검색 가능

미디어 연구 활용: 뉴스 사진 프레이밍 분석, 텍스트-사진 일치도 측정, 이미지 검색

벡터 공간과 코사인 유사도






```python
import numpy as np
def cosine_sim(x, y):
    return np.dot(x,y) /
    (np.linalg.norm(x) * np.linalg.norm(y))
# a=[1,2,3], b=[2,4,6] → 1.0 (같은 방향)
  a, c=[3,-1,0] → 0.08 (무관)
```

https://wikidocs.net/24603

실습: 코사인 유사도

```
import numpy as np

def cosine_sim(x, y):
    return np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))


# 벡터 정의
a = np.array([1, 2, 3])
b = np.array([2, 4, 6]) # 같은 방향
c = np.array([3, -1, 0]) # 거의 무관

# 결과 출력
print("a vs b:", cosine_sim(a, b)) # ≈ 1.0
print("a vs c:", cosine_sim(a, c)) # ≈ 0.08
```

실습: Sentence-BERT로 문장 테스트!

```bash
#bash (패키지 설치)
pip install -U sentence-transformers
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

```python
#python
import numpy as np
from sentence_transformers import SentenceTransformer, util

# 1. 모델 로드 (CPU 명시)
model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2') #빠르고 가벼운 모델
# 2. 문장
titles = [
    "경제 성장 정책 발표",    # A
    "경제 침체 대책 논의",    # B
    "야구 선수 홈런 기록"     # C
]

# 3. 임베딩 생성
vectors = model.encode(titles)
print("벡터 shape:", vectors.shape)    # (3, 768)

# 4. 코사인 유사도
cos_sim_matrix = util.cos_sim(vectors, vectors)

print("\n코사인 유사도 행렬:")
print(cos_sim_matrix)
```